

6 EE PROJECTS THAT EAVESDROP ON THE SKY

Aircraft. Ships. Meteors. Weather balloons. The sensors around you.
All receive-only, all legal. One \$30 radio. Real RF.

- | | |
|--|-------------|
| 1. ADS-B Aircraft Radar — a live map of every plane overhead | \$30 |
| 2. TPMS Car-ID Sniffer — see what your own car broadcasts | \$30 |
| 3. Radio Meteor Detector — a ping for every shooting star | \$30 |
| 4. AIS Ship Tracker — live positions of every ship in range | \$35 |
| 5. 433 MHz Sensor Decoder — read the ISM chatter around you | \$25 |
| 6. Radiosonde Tracker — chase a weather balloon from 30 km up | \$35 |

PROJECT 1

ADS-B Aircraft Radar

Build a live map of every plane overhead — callsign, altitude, speed — pulled out of the air onto your screen

Cost: ~\$30

Difficulty:
Beginner-Intermediate

Build Time: 1 weekend

Why This Matters

Every airliner constantly broadcasts its identity, GPS position, altitude, and velocity in the clear on 1090 MHz — a system called ADS-B that is replacing radar for air-traffic control worldwide. There is no encryption and no subscription: with a \$30 software-defined radio and a small antenna, you can receive those broadcasts directly and plot every aircraft in range on a live map, exactly like the professional flight-tracking sites (which are largely built from volunteer receivers just like the one you will build). It is the perfect first SDR project — the signal is strong, always present, and the payoff is immediate and genuinely impressive: a real-time radar picture of the sky above you, decoded by hardware you assembled.

What this proves to a recruiter: You understand SDR fundamentals, pulse-position-modulated digital signals (Mode S), antenna selection for a specific band, and real-time packet decoding. Receiving and decoding a live aviation datalink is a concrete, demonstrable RF skill that maps straight onto avionics, surveillance, and wireless-systems work.

→ **The job this maps to:** air-traffic / surveillance systems, avionics, and SDR/RF engineering — flight-tracking networks, air-navigation service providers, and aerospace firms.

ADS-B Aircraft Radar — Receive Chain



Aircraft broadcast position + ID on 1090 MHz (ADS-B / Mode S), unencrypted
Decoder extracts each message → plots callsign, altitude, and track on a live map

How It Works

- **ADS-B on 1090 MHz:** Aircraft transponders broadcast 'extended squitter' messages roughly twice a second containing the 24-bit ICAO address, GPS latitude/longitude, altitude, speed, and callsign. The signals are unencrypted and standardised worldwide, so any compatible receiver can decode them.
- **Pulse-position modulation:** Mode S encodes bits as the position of short pulses within each microsecond — a 1 Mbit/s pulse-position-modulated stream. The decoder samples fast enough to distinguish pulse timing, reconstruct the message bits, and check them with a CRC.
- **Antenna tuned to 1090 MHz:** At 1090 MHz a quarter-wave is only ~6.9 cm, so a small ground-plane or 'spider' antenna works and a collinear gives more range. Antenna length and placement (high, clear of obstructions) matter far more than fancy electronics on this band.
- **LNA and filter help range:** Distant aircraft are weak and the band sits near strong cellular and pager interference. A 1090 MHz SAW filter plus a low-noise amplifier at the antenna dramatically extends range — from tens to a couple hundred kilometres line-of-sight.
- **Decode and map (dump1090):** Software like dump1090 demodulates the Mode S stream, validates each message by CRC, and converts the position fields into latitude/longitude. It serves a live browser map showing every aircraft's callsign, altitude, heading, and trail — a real-time radar display you built.

Key Design Decisions

- **Why does ADS-B need no decryption?** ADS-B was designed as an open air-traffic-control broadcast so any aircraft or ground station can see the picture. Privacy of the broadcast was deliberately traded for universal situational awareness — which is exactly why a hobbyist can receive it legally.
- **Why is antenna height the biggest factor?** 1090 MHz is line-of-sight; the radio horizon, not transmit power, sets your range. Raising the antenna a few metres and clearing obstructions adds far more range than any amplifier, because you are literally seeing farther over the curve of the Earth.
- **Why a filter before the amplifier?** Strong nearby signals (cellular, pagers) can overload a broadband LNA and desensitise it. A SAW filter centred on 1090 MHz ahead of the LNA rejects those, so the amplifier's gain goes to the signal you want instead of amplifying interference.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2 (1090 MHz capable)	1	\$30
1090 MHz antenna	Quarter-wave ground plane or collinear	1	\$0-8
1090 MHz LNA + SAW	Optional, for range	1	\$10
Coax + SMA	Low-loss feedline	1	\$5
Mount / mast	Get the antenna high	1	\$5
Raspberry Pi / laptop	Runs dump1090	1	—

Build Steps

Plug in and scan: Install the RTL-SDR drivers and a general SDR app. Tune to 1090 MHz and confirm you see brief bursts of activity — those are aircraft squitters. This proves the dongle and antenna work before you add anything.

Build a 1090 MHz antenna: Make a quarter-wave ground-plane (a 6.9 cm vertical with four radials) or a coaxial collinear for more gain. Mount it as high and clear as possible — a window or rooftop. Connect with low-loss coax.

Add the LNA + filter: For more range, fit a 1090 MHz SAW-filtered LNA at the antenna and power it via the dongle's bias-tee. Verify it is the 1090 MHz part — a wrong-band LNA will reduce performance.

Run dump1090: Install dump1090 (or readsb) on a Raspberry Pi or laptop, start it, and open the web interface. Decoded aircraft appear with ICAO address, callsign, altitude, and speed.

Map and log: Enable the live map view and watch aircraft tracks build in real time. Log a day of traffic and plot your maximum range and aircraft count — or feed your data to a tracking network and get a free membership in return.

Interview Questions:

- What information does an ADS-B extended squitter contain, and why is it unencrypted?
- How does Mode S pulse-position modulation encode bits?
- Why is antenna height the dominant factor in your reception range at 1090 MHz?
- Why place a SAW filter before the LNA in this receive chain?
- How is each message validated, and what does the CRC protect against?
- How would you estimate the maximum range to an aircraft at a given altitude?

PROJECT 2

TPMS Car-ID Sniffer

Reveal that every car driving past silently broadcasts a unique ID — it is not your phone tracking you, it is your tires

Cost: ~\$30

Difficulty: Intermediate

Build Time: 1 weekend

Why This Matters

Every modern car has tire-pressure sensors that periodically transmit on 315 MHz (North America) or 433 MHz (Europe) — and each transmission includes a unique sensor ID along with pressure and temperature. They are unencrypted and broadcast whenever the wheels turn, which means a cheap radio can log them. Researchers have shown TPMS IDs can be used to identify and track vehicles, and this project lets you see that privacy gap with your own eyes: park near a road, decode the packets, and watch unique IDs roll past. It is a striking lesson in how much our everyday devices leak — and exactly the kind of wireless-security insight that automotive and IoT-security teams hire for.

What this proves to a recruiter: You understand sub-GHz ISM transmission, OOK/FSK demodulation, reverse-engineering an undocumented packet format, and the privacy implications of unencrypted wireless — the core skill set of automotive RF and wireless-security engineering.

→ **The job this maps to:** automotive RF, wireless security / penetration testing, and embedded-systems roles — automakers, Tier-1 suppliers, and IoT-security firms.

Legal & ethics: This is a receive-only demonstration of how much your own car leaks — best done logging your own vehicles. Building a system to track other people by their TPMS IDs is a privacy violation (and in many places regulated). Observe to understand the problem; do not operate it as a surveillance tool.

TPMS Car-ID Sniffer — Decode Chain



Tire sensors transmit ID + pressure + temperature on 315/433 MHz, unencrypted
Demodulate and decode the packet → a unique ID appears for every passing wheel

How It Works

- **TPMS transmissions:** Each sensor wakes periodically (or on rotation) and transmits a short burst with its unique 28-32 bit ID, the tire pressure, temperature, and a checksum. The bursts are unencrypted and repeat every few seconds to a minute.
- **315 vs 433 MHz:** North American cars typically use 315 MHz; European and many other markets use 433.92 MHz ISM. You tune the SDR to the right band for your region — the RTL-SDR covers both easily.
- **OOK and FSK modulation:** TPMS sensors use on-off keying or frequency-shift keying with Manchester or differential encoding. The decoder demodulates the chosen scheme, recovers the bitstream, then aligns to the packet's preamble and sync word.
- **Reverse-engineering the format:** Different manufacturers pack the fields differently. Tools like `rtl_433` already know dozens of TPMS protocols; decoding a new one means finding the preamble, the ID field, and how pressure and temperature are scaled — real protocol reverse-engineering.
- **Seeing the privacy gap:** Because the ID is static and unencrypted, the same car produces the same ID every time it passes. Logging these is exactly how researchers demonstrated vehicle tracking — and seeing your own car's IDs appear makes the abstract risk concrete.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2	1	\$30
Antenna	Tuned for 315 or 433 MHz (telescopic)	1	\$5
Quarter-wave whip	Cut for your band (alt antenna)	1	\$2
315/433 SAW LNA	Optional, for weak bursts	1	\$10
Coax + SMA	Short feedline	1	\$3
USB extension	Position the dongle near a window	1	\$3
Laptop / Pi	Runs rtl_433	1	—

Build Steps

Identify your band: Find whether your region/car uses 315 MHz or 433 MHz, tune the RTL-SDR there, and look for short periodic bursts when a car is nearby or you roll your own.

Set up rtl_433: Install rtl_433, the open-source decoder that already understands many TPMS protocols. Run it on your band; as cars pass or your own car rolls, decoded packets with IDs, pressures, and temperatures print.

Confirm with your own car: Roll your own vehicle and watch its four sensor IDs appear, each with a plausible pressure reading. Let a little air out of one tire and watch that sensor's value drop — proof you are decoding real telemetry.

Log passing traffic (ethically): Park near a road and log the stream for a few minutes. Note how each car contributes a cluster of repeating IDs — the dataset that demonstrates the tracking risk. Keep it to understanding the problem, not profiling people.

Reverse a new protocol (bonus): If you hit a sensor rtl_433 does not recognise, capture the raw burst, find the preamble and sync, and work out the ID and pressure fields. Decoding an unknown format from scratch is a strong write-up.

Interview Questions:

- What does a TPMS transmission contain, and why is each sensor identifiable?
- Why do North American and European TPMS systems use different frequencies?
- What is the difference between OOK and FSK, and how do you demodulate each?
- How would you reverse-engineer an unknown sensor's packet format?
- What are the privacy implications of static, unencrypted device identifiers?
- How could a manufacturer redesign TPMS to prevent tracking while keeping it cheap?

PROJECT 3

Radio Meteor Detector

Catch a ping for every shooting star as it ionizes the upper atmosphere — even in daylight or solid cloud

Cost: ~\$30

Difficulty: Intermediate

Build Time: 1 weekend

Why This Matters

When a meteor streaks through the upper atmosphere it leaves a trail of ionized gas that briefly reflects radio waves — and you can hear it. By tuning to a powerful distant VHF transmitter that is normally below your horizon (a dedicated radar like GRAVES, or a far-off TV or FM carrier), you receive nothing... until a meteor trail bounces that signal over the horizon to your antenna, producing a sudden 'ping.' This is forward-scatter meteor detection, and it works day or night, through clouds, even when you cannot see a thing. You are detecting objects from space by their effect on radio propagation — real radio-astronomy technique, built from a \$30 dongle and a simple antenna.

What this proves to a recruiter: You understand weak-signal reception, the radio horizon and forward-scatter propagation, narrowband detection and spectrograms, and distinguishing real events from noise. It is a genuine radio-astronomy instrumentation skill that maps onto research, defence, and DSP roles.

→ **The job this maps to:** radio-astronomy instrumentation, weak-signal RF, and DSP roles — research institutions, observatories, and signal-processing teams.

Radio Meteor Detector — Forward Scatter



A distant VHF carrier is normally below the horizon → you hear nothing
A meteor's ionized trail reflects it to you → a sudden ping shows on the spectrogram

How It Works

- **Forward-scatter geometry:** You point a directional antenna toward a powerful VHF transmitter hundreds to ~1500 km away — far enough that it is below your horizon and you normally receive nothing on its frequency. The transmitter and you form a 'bistatic' pair waiting for something to bounce the signal across.
- **Meteors make a mirror:** A meteoroid hitting the atmosphere at tens of km/s ionizes a column of air ~80-120 km up. That plasma trail briefly reflects VHF radio: for a fraction of a second to a few seconds, the distant transmitter's carrier is scattered down to your antenna — a 'ping.'
- **Choosing the transmitter:** Dedicated beacons like the French GRAVES radar (143.05 MHz) are ideal where receivable; elsewhere, distant analog TV or FM carriers, or amateur beacons, serve the same role. You tune precisely to the carrier and look for reflections, not the direct signal.
- **Detecting the ping:** Software (SDR# plus a spectrogram, or a dedicated detector) shows a waterfall of the narrow frequency around the carrier. A meteor appears as a sudden bright trace — often Doppler-shifted as the trail drifts — lasting from a fraction of a second to many seconds for a bright fireball.
- **Counting and timing:** Logging pings over hours reveals meteor activity, including the dramatic rise during showers like the Perseids. Because radio does not care about daylight or clouds, you detect meteors a visual observer never could — including the daytime showers.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2	1	\$30
VHF Yagi antenna	~3-5 element for the chosen band	1	\$10-15
Coax + SMA	Low-loss feedline	1	\$5
VHF LNA	Optional, low-noise for weak pings	1	\$10
Mast / mount	To aim the Yagi	1	\$5
Laptop	Runs SDR + spectrogram / detector	1	—

Build Steps

Pick a transmitter + frequency: Choose a powerful VHF source beyond your horizon — GRAVES at 143.05 MHz if you are in range, otherwise a distant TV/FM carrier or beacon. Note its exact frequency and compass bearing from you.

Aim a Yagi at it: Build or buy a small VHF Yagi for that frequency and point it toward the transmitter's bearing, tilted up slightly toward the reflection region. Directionality rejects local noise and favours the scatter path.

Tune precisely to the carrier: In your SDR software, tune exactly to the transmitter frequency with a narrow bandwidth. With no meteor you should see only noise (the direct signal is over the horizon). This 'silent' baseline is what makes pings obvious.

Watch the waterfall: Open a spectrogram zoomed tight on the carrier. When a meteor passes, a bright streak appears — sometimes a sharp dot, sometimes a long Doppler-curved trace for a bright event. Confirm a few by ear (a sudden whistle or ping).

Log and correlate: Run an automatic detector or log manually over a night, counting pings per hour. Compare a normal night to a meteor-shower peak and plot the rate — your own radio meteor-activity curve.

Interview Questions:

- What physically reflects the radio signal during a meteor detection?
- Why must the reference transmitter be beyond your horizon for forward scatter to work?
- Why can radio detect meteors in daylight and through clouds when optical cannot?
- What causes the Doppler shift seen on a meteor ping?
- How do you distinguish a genuine meteor ping from aircraft reflections or interference?
- How would you automate counting and characterising pings over a long observation?

PROJECT 4

AIS Ship Tracker

The maritime cousin of ADS-B — live positions, names, headings, and cargo of every ship in range

Cost: ~\$35

Difficulty:
Beginner-Intermediate

Build Time: 1 weekend

Why This Matters

Ships continuously broadcast their identity, GPS position, course, speed, and cargo type on marine VHF (161.975 and 162.025 MHz) using a system called AIS — the maritime equivalent of aircraft ADS-B. It is unencrypted and required on most large vessels, so with an SDR tuned to the marine band you can plot live ship traffic on a map: tankers, ferries, fishing boats, naval auxiliaries, each with its name and destination. If you are anywhere near a coast, port, or large river, the result is a busy real-time picture of maritime traffic — and the same volunteer-receiver model behind the public ship-tracking sites.

What this proves to a recruiter: You understand VHF reception, GMSK demodulation, the AIS/NMEA message format, and multi-channel decoding. Receiving and plotting a live maritime datalink is a concrete RF and decoding skill directly relevant to maritime, surveillance, and wireless-systems work.

→ **The job this maps to:** maritime / navigation systems, coastal surveillance, and SDR/RF roles — shipping, port authorities, defence, and tracking networks.

AIS Ship Tracker — Receive Chain



Ships broadcast position + ID on marine VHF (161.975 / 162.025 MHz), unencrypted
Demodulate both AIS channels → decode NMEA → plot every vessel on a live map

How It Works

- **AIS on marine VHF:** The Automatic Identification System broadcasts on two channels near 162 MHz. Vessels send their MMSI identity, GPS position, course, speed, and (in static messages) ship name, type, and destination, on a self-organising time-division schedule so many ships share the channel.
- **Two channels at once:** AIS uses 161.975 MHz (channel A) and 162.025 MHz (channel B). A capable receiver decodes both simultaneously — the RTL-SDR's bandwidth easily spans both, and the decoder splits and demodulates each.
- **GMSK modulation:** AIS uses Gaussian minimum-shift keying at 9600 bps — a compact, constant-envelope FSK variant. The decoder demodulates GMSK, recovers the HDLC-framed packets, and checks each with a CRC.
- **NMEA decoding:** Valid packets are turned into NMEA AIS sentences (the same format chartplotters use). Software like AIS-catcher or rtl-ais parses these into position and vessel-info records.
- **Plot the traffic:** Feed the decoded positions to a map (AIS-catcher's web UI or a plotter). Ships appear with names, headings, and tracks; static messages fill in type and destination. Near a coast you will see a constantly updating maritime picture.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2	1	\$30
Marine VHF antenna	Tuned ~162 MHz (or wideband VHF)	1	\$8
VHF LNA + filter	Optional, for range	1	\$10
Coax + SMA	Low-loss feedline	1	\$5
Mast / mount	Height improves range	1	\$5
Laptop / Pi	Runs AIS-catcher / rtl-ais	1	—

Build Steps

Set up a VHF antenna: Use a marine-band (~162 MHz) vertical or a wideband VHF antenna, mounted as high and clear as possible — ideally with sea view. Range is line-of-sight, so height and a clear path to the water matter most.

Install an AIS decoder: Install AIS-catcher (or rtl-ais), configure it for the RTL-SDR and the two AIS channels, start it, and watch decoded messages appear in the console.

Confirm both channels: Verify you are decoding traffic on both 161.975 and 162.025 MHz — many decoders show per-channel message counts. Strong nearby vessels should appear within minutes if you are near water.

Open the live map: Enable the web map and watch vessels populate with MMSI, name, course, and speed. Static messages will fill in ship type and destination as they are received.

Log and analyse: Record a few hours of traffic and analyse it — busiest vessel types, your maximum decode range, message rates by channel. Optionally feed a tracking network for a free upgraded account.

Interview Questions:

- What information do AIS position and static messages contain?
- Why does AIS use two channels, and how does the receiver handle both?
- What is GMSK, and why is a constant-envelope modulation useful here?
- How does AIS coordinate many ships sharing the same channel (SOTDMA)?
- Why is antenna height and sea view the dominant range factor?
- How does AIS compare to radar for collision avoidance — strengths and gaps?

PROJECT 5

433 MHz Sensor Decoder

Decode the invisible chatter of the ISM band — weather stations, doorbells, thermometers, your own smart-home sensors

Cost: ~\$25

Difficulty:
Beginner-Intermediate

Build Time: 1 weekend

Why This Matters

The 433 MHz ISM band is full of chatter you never notice: wireless weather stations, remote thermometers, soil-moisture sensors, doorbells, garage remotes, and cheap smart-home gadgets all transmit there in the clear. Point an SDR at it and an open-source decoder turns that noise into readable data — temperatures, humidity, button presses, sensor IDs — from devices all around your neighbourhood and inside your own home. It is the most approachable possible introduction to IoT reverse-engineering, and the natural gateway to building your own sensor network or auditing what your devices leak. The same skills underpin IoT product development and wireless security.

What this proves to a recruiter: You understand sub-GHz ISM communication, OOK/FSK demodulation, packet framing and checksums, and decoding undocumented device protocols. This is exactly the wireless skill set behind IoT product engineering and connected-device security.

→ **The job this maps to:** IoT / embedded wireless, home-automation, and wireless-security roles — IoT product companies, smart-home platforms, and security teams.

433 MHz Sensor Decoder — ISM Capture



Weather stations, remotes, and sensors broadcast on the 433 MHz ISM band, unencrypted
Demodulate and frame the packets → rtl_433 turns them into readable values

How It Works

- **The 433 MHz ISM band:** 433.92 MHz is a licence-free band where countless low-power devices transmit short, simple messages. Because the band is shared and cheap, manufacturers use unencrypted OOK/FSK with basic framing — easy to receive and decode.
- **OOK and FSK:** Most cheap sensors use on-off keying (the carrier blips on and off) or simple FSK. The decoder demodulates the carrier into a raw pulse train, then interprets pulse widths as bits using each protocol's encoding (PWM, Manchester, and so on).
- **Packet framing and IDs:** A typical message has a preamble, a device ID, the payload (temperature, humidity, state), and a checksum. The ID lets you tell your neighbour's weather station from your own; the checksum lets the decoder reject corrupted packets.
- **rtl_433 does the heavy lifting:** rtl_433 is an open-source decoder that recognises hundreds of device protocols out of the box. Run it and it auto-identifies known devices, printing manufacturer, model, ID, and decoded values as JSON you can log or pipe anywhere.
- **Decode the unknown:** When you hit a device rtl_433 does not know, you capture the raw signal, study the pulse pattern, and work out the encoding and fields yourself — a clean, self-contained protocol reverse-engineering project that is genuinely impressive on a resume.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2	1	\$25
433 MHz antenna	Quarter-wave whip (~17 cm) or telescopic	1	\$4
Coax + SMA	Short feedline	1	\$3
433 MHz LNA	Optional, for weak/distant devices	1	\$8
USB extension	Position the dongle / antenna	1	\$3
Laptop / Pi	Runs rtl_433	1	—

Build Steps

Tune to 433.92 MHz: Install the RTL-SDR tools and tune to 433.92 MHz in an SDR app. Trigger a known device (press a wireless doorbell or remote) and watch the burst appear — confirming your setup hears the band.

Run rtl_433: Install and run rtl_433 on 433.92 MHz. Within minutes it will start auto-decoding nearby weather stations, thermometers, and sensors, printing model names, IDs, and values.

Identify your own devices: Cross-reference the decoded IDs with your own gadgets — your weather station's temperature should match reality. This separates your devices from the neighbourhood's chatter.

Log to a dashboard: Pipe rtl_433's JSON output to a simple logger or home-automation system (MQTT, InfluxDB/Grafana). Now you are capturing live sensor data into your own dashboard — the start of a DIY sensor network.

Reverse an unknown device: Find a device rtl_433 cannot decode, capture its raw pulses (-A analyzer mode), determine the encoding and fields, and write a decoder. Document the process — a strong, focused portfolio piece.

Interview Questions:

- Why is the 433 MHz band so crowded with simple unencrypted devices?
- What is on-off keying, and how do you turn a pulse train into bits?
- What fields make up a typical sensor packet, and what does the checksum do?
- How would you reverse-engineer a device rtl_433 does not recognise?
- What security and privacy issues arise from unencrypted ISM sensors?
- How would you build a reliable home sensor network on top of these decodes?

PROJECT 6

Radiosonde Tracker & Recovery

Lock onto a weather balloon's telemetry as it falls from 30 km up — then go physically chase it down

Cost: ~\$35

Difficulty: Intermediate

Build Time: 1-2 weekends

Why This Matters

Twice a day, from hundreds of sites worldwide, weather agencies launch balloons carrying radiosondes — small instrument packages that transmit temperature, humidity, pressure, and GPS position as they rise to ~30 km, then parachute back to Earth. Those transmissions (around 400-406 MHz) are unencrypted, so you can receive a sonde's live telemetry, plot its track, and — using its GPS position and a descent model — predict where it will land and go recover it. It is a genuine treasure hunt powered by RF: decode a real aerospace telemetry stream, do the tracking and prediction math, and end up holding the instrument you tracked from the edge of space. Recovered sondes are also reusable for your own high-altitude projects.

What this proves to a recruiter: You understand telemetry decoding, GNSS position data, weak-signal VHF/UHF reception with directional antennas, and trajectory prediction. Tracking a live aerospace telemetry payload from launch to landing is a standout, end-to-end RF-and-data skill for aerospace and telemetry roles.

→ **The job this maps to:** telemetry / GNSS, aerospace / avionics, and RF-systems roles — weather agencies, aerospace firms, and high-altitude ballooning teams.

Recovery etiquette: Recovering a landed radiosonde is legal in most places (they are considered abandoned), but the chase is the risky part: do not trespass, do not enter roads or rail, and respect private property and local rules. Use the telemetry to predict the landing site, then knock and ask before crossing anyone's land.

Radiosonde Tracker — Telemetry & Recovery



Weather-balloon sonde transmits GPS + sensors on ~400-406 MHz, unencrypted
Decode telemetry → plot track and predict landing → go recover the payload

How It Works

- **Radiosonde telemetry:** A radiosonde (commonly a Vaisala RS41 or Graw type) transmits a continuous data frame with its GPS latitude, longitude, and altitude plus temperature, humidity, and pressure, on a frequency in the 400-406 MHz meteorological band. Each launch site uses a known frequency you can look up.
- **Receiving the signal:** As the balloon climbs it becomes line-of-sight over a huge area, so reception is easy at altitude; the hard part is the low-altitude descent and landing. A directional UHF antenna (a small Yagi) and an LNA help you hold the signal as the sonde drops behind terrain.
- **Decoding the frame:** Software like `radiosonde_auto_rx` demodulates the sonde's FSK telemetry, decodes the frame format for that sonde type, validates it, and outputs live position and sensor data — often uploading to a tracking map automatically.
- **Predicting the landing:** Once you have the descent trajectory and the sonde's current position and altitude, a descent model (accounting for wind drift under the parachute) predicts the landing spot. The prediction tightens as the sonde nears the ground.
- **The recovery:** With a predicted landing zone, you drive toward it, switch to a handheld Yagi for the final direction-finding, and home in on the last-known position. Recovering the payload is the tangible payoff — and the sonde can be reflashed for your own balloon or tracking experiments.

Bill of Materials

Component	Part / Spec	Qty	~Cost
RTL-SDR dongle	RTL2832U + R820T2	1	\$30
UHF Yagi antenna	~400 MHz, 3-5 element (recovery DF)	1	\$12
Wideband whip	For easy reception at altitude	1	\$4
400 MHz LNA	Optional, low-altitude reception	1	\$10
Coax + SMA	Feedline	1	\$5
Phone / GPS	Navigate to predicted landing	1	—
Laptop / Pi	Runs auto_rx	1	—

Build Steps

Find a launch + frequency: Look up your nearest radiosonde launch site, its schedule (often ~00:00 and 12:00 UTC), and the transmit frequency. Sites and live sondes are listed on public tracking maps.

Set up auto_rx: Install radiosonde_auto_rx on a Raspberry Pi or laptop with the RTL-SDR. Configure your frequency range; it will scan, detect a sonde, lock on, and decode its telemetry automatically.

Track the flight: Once decoding, watch the sonde's live position, altitude, and sensor data. Follow it up to burst altitude (~30 km) and into descent. auto_rx can upload to a map so you see the whole trajectory.

Predict the landing: Use the descent prediction (built into the tracking tools) to estimate the landing zone from current position, altitude, and wind. Refine as it drops — the cone narrows quickly near the ground.

Drive out and recover: Head toward the predicted site, switch to a handheld Yagi to direction-find the final signal, and locate the payload. Respect property and access rules. Reflash the recovered sonde for your own projects if you like.

Interview Questions:

- What data does a radiosonde transmit, and in what frequency band?
- Why is the descent and landing the hardest part to receive, and how do you compensate?
- How is the telemetry frame demodulated and validated?
- How do you predict a landing site from position, altitude, and wind?
- How does a Yagi help with final direction-finding on the ground?
- What GNSS and coordinate concepts are involved in plotting the track?

The sky is full of signals nobody's listening to. One \$30 radio, and you are.

Aircraft, ships, weather balloons, meteors, and the sensors all around you are broadcasting constantly, in the clear, to anyone with a receiver. With a single software-defined radio and the right antenna, you can pull all of it out of the air and decode it yourself — a live radar of the sky, a map of the seas, a ping for every shooting star. The hardware is cheap; the skill is in the antennas, the weak-signal reception, and the protocol decoding.

And here is the throughline that makes it more than a hobby: every one of these is receive-only and legal, and that restraint is exactly what real RF and security engineers practise. Decoding what is already public, understanding what leaks, and respecting what is private is the professional version of this craft. SDR, aerospace surveillance, maritime systems, and signals work all hire people who can do precisely this.

Build even one of these. Screenshot the live map, log the data, plot your range. Put it on your GitHub with a proper README and a note on the legal, receive-only scope. You will have a portfolio piece that proves you can pull real signals out of the air — and the judgment to do it responsibly.

Follow @1nska on Instagram for build logs, new guides, and projects that listen to the whole sky.

— Enes (@1nska) • Electrical Engineer